Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

1. The number of pointers grounded in a tree of n nodes is

a) n + 1

b) n - 1

c) n

d) 2n

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

1. The number of pointers grounded in a tree of n nodes is

a) n + 1

b) n - 1

c) n

d) 2n

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

2. Left of new node inserted in a threaded binary search tree(for inorder traversal)

a) Is always a structural pointer

b) Is a thread and gets the value of left of parent

c) Is a thread and gets the value of right of parent

d) Is always grounded

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

2. Left of new node inserted in a threaded binary search tree(for inorder traversal)

a) Is always a structural pointer

b) Is a thread and gets the value of left of parent

c) Is a thread and gets the value of right of parent

d) Is always grounded

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

3. Lthread = 1 implies that the left is a thread. Rthread = 1 implies that the right is a thread. Initializing lthread and rthread of a new node

a) to 0 is preferred as left and right pointers of a new node are always structural

b) to 1 is preferred as left and right pointers of a new node are always threads

c) to 0 is preferred as left and right pointers of a new node are always threads

d) to 1 is preferred as left and right pointers of a new node are always structural

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

3. Lthread = 1 implies that the left is a thread. Rthread = 1 implies that the right is a thread. Initializing lthread and rthread of a new node

a) to 0 is preferred as left and right pointers of a new node are always structural

b) to 1 is preferred as left and right pointers of a new node are always threads

c) to 0 is preferred as left and right pointers of a new node are always threads

d) to 1 is preferred as left and right pointers of a new node are always structural

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

4. Lthread = 1 and Rthread = 1 implies that

a) the node is a leaf

b) the node is an internal node

c) The node should be the root

d) Nothing can be said about the node

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

4. Lthread = 1 and Rthread = 1 implies that

a) the node is a leaf

b) the node is an internal node

c) The node should be the root

d) Nothing can be said about the node

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

5. When a new is added, the right of the node

a) Is a thread and refers to its parent

b) Is structural and refers to its parent

c) Is a thread and refers to itself

d) Is structural and refers to itself

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Threaded Binary Tree

5. When a new is added, the right of the node

a) Is a thread and refers to its parent

b) Is structural and refers to its parent

c) Is a thread and refers to itself

d) Is structural and refers to itself

© Confidential Information of PES University. All Rights Reserved - 2026

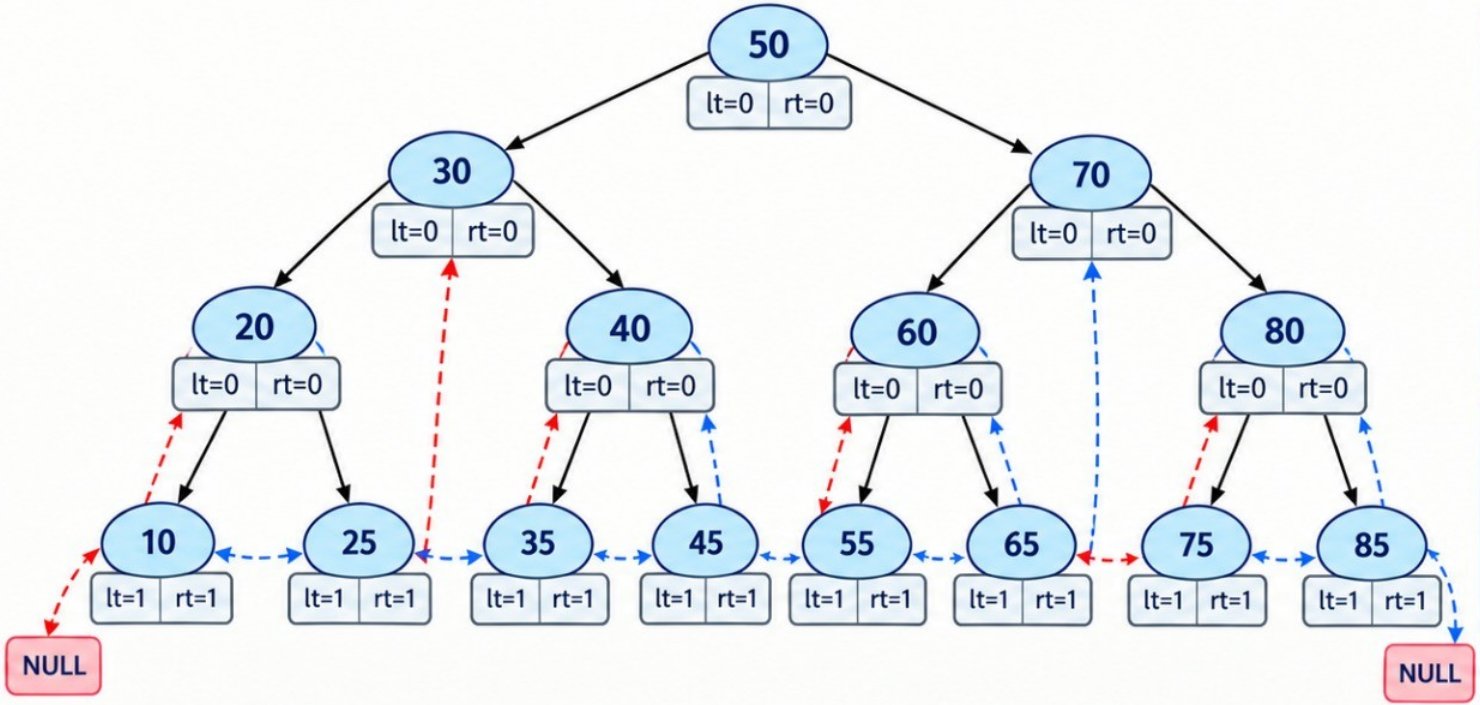
Threaded Binary Tree

- In a binary tree of n nodes, there are 2n pointers
- A tree has n-1 edges.
- n + 1 pointers are grounded

- Make the right pointers which are normally grounded to point the successor in some traversal
- Make the left pointers which are normally grounded to point the predecessor in some traversal
- Avoid recursive calls while traversing – so efficient in time
- Requires extra space to indicate whether a pointer is a structural pointer or a thread
- Inorder traversal selected in our discussion

© Confidential Information of PES University. All Rights Reserved - 2026

Fully Threaded Binary Tree (with Inorder Threads)



Legend

= Child link

= Left thread (Inorder predecessor)

= Right thread (Inorder successor)

key | lt | rt = Node structure (lt = 0/1, rt = 0/1)

Node structure (in code)

```
typedef struct node {
    int key;
    struct node *left;
    struct node *right;
    int lthread;
    int rthread;
}
typedef struct node node_t;
```

Inorder sequence (via threads)

NULL ↔ 10 → 20 → 25 → 30 → 35 → 40 → 45 → 50 → 55 → 60 → 65 → 70 → 75 → 80 → 85 → NULL

(Threads connect nodes in inorder sequence)

Threaded Binary Tree

- Structure:

```
struct node
{
        int key_;
        struct node *left_;
        struct node *right_;
        int lthread_; // 1 if left_ is a thread
        int rthread_; // 1 if right_ is a thread
};
typedef struct node node_t;
```

```
struct tree
{
        node_t *root_;
};
typedef struct tree tree_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

- interface:

```
void init(tree_t *ptr_tree);
void deinit(tree_t *ptr_tree);
void disp(tree_t *ptr_tree);
void insert(tree_t *ptr_tree, int key);
```


© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

- initialize:

```
void init(tree_t *ptr_tree)
{
        ptr_tree->root_ = NULL;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

- insert: helper function

```
static node_t* create_node(int key)
{
        node_t *temp = (node_t*)malloc(sizeof(node_t));
        if(temp == NULL)
        {
                printf("no more memory\n"); exit(1);
        }
        temp->key_ = key;
        temp->left_ = temp->right_ = NULL; // should be changed on insertion
        temp->lthread_ = temp->rthread_ = 1; // should be threads
        return temp;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

insert:
Insert_key is not available to the client
This knows the node structure.

```
void insert(tree_t *ptr_tree, int key)
{
        ptr_tree->root_ = insert_key(ptr_tree->root_, key);
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

insert:

```
static node_t* insert_key(node_t* root, int key)
{
   node_t *temp = create_node(key);
   node_t *parent = NULL;
   node_t *curr = root;
   while (curr != NULL)
   {
      parent = curr;
      if (key < curr->key_)
      {
         if (curr->lthread_ == 0)
                curr = curr->left_;
         else
                 break;
      }
```

```
      else
      {
            if (curr->rthread_ == 0)
                  curr = curr->right_;
            else
                  break;
      }
   } // of while
```

if key is less than that of the root move left until there is left subtree. Lthread = 1 => no left subtree

If key is not less than that of the root move right until There is no right subtree. Rthread = 1 => no right subtree

© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

insert: continued

```
if (parent == NULL)
  {
     return temp; // empty tree case
  }
   /* Insert as left child */
  if (key < parent->key_)
  {
     temp->left_ = parent->left_;
     temp->right_ = parent;

     parent->left_ = temp;
     parent->lthread_ = 0;
  }
```

temp->right_ is a thread and refer to its parent. parent->left_ refers to temp and is no more a thread.

```
 /* Insert as right child */
   else
   {
      temp->right_ = parent->right_;
      temp->left_ = parent;

      parent->right_ = temp;
      parent->rthread_ = 0;
   }

   return root;
}
```

Insertion on the right subtree is similar.

© Confidential Information of PES University. All Rights Reserved - 2026

Threaded Binary Tree

Inorder traversal :

```
 void disp(tree_t *ptr_tree)
{
                printf("Tree : ");
                inorder(ptr_tree->root_);
                printf("\n");
}
```

Helper function:

```
// returns the node with no left subtree
static node_t *leftmost(node_t *temp)
{
   if (temp == NULL)
      return NULL;

   while (temp->lthread_ == 0)
      temp = temp->left_;

   return temp;
}
```

```
static void inorder(node_t *root)
{
  // start from the leftmost node in the tree
   node_t *temp = leftmost(root);

   while (temp != NULL)
   {
      printf("%d ", temp->key_);
     // find the inorder successor
    // follow the right thread if the pointer is a thread
      if (temp->rthread_ == 1)
         temp = temp->right_;
      else
     // otherwise find the leftmost of its right subtree
         temp = leftmost(temp->right_);
   }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)